키 교환 및 암호화 메커니즘 문서화

개요

이 문서는 Rust libp2p에서 구현된 키 교환 및 암호화 메커니즘을 상세히 분석합니다. libp2p는 P2P 네트워크에서 안전한 통신을 위해 다양한 암호화 프로토콜과 키 관리 시스템을 제공합니다.

1. 신원 키 관리 시스템 (Identity Key Management)

1.1 지원되는 키 타입

libp2p는 다음과 같은 암호화 키 타입을 지원합니다:

```
pub enum KeyType {
Ed25519, // Edwards-curve Digital Signature Algorithm
RSA, // Rivest-Shamir-Adleman
Secp256k1, // ECDSA over secp256k1 curve
Ecdsa, // Elliptic Curve Digital Signature Algorithm
}
```

1.2 Ed25519 키 구현

Ed25519는 libp2p에서 권장되는 기본 키 타입입니다:

키 생성:

```rust
impl Keypair {
pub fn generate() -> Keypair {
Keypair::from(SecretKey::generate())
}
}


impl SecretKey {
pub fn generate() -> SecretKey {
let mut secret = ed25519::SecretKey::default();
rand::rngs::OsRng.fill_bytes(&mut secret);
SecretKey(secret)
}
}
```

키 특징:

- 32바이트 비밀키, 32바이트 공개키
- 64바이트 서명 생성
- 빠른 서명 생성 및 검증
- 강력한 보안성 (128비트 보안 수준)

1.3 키 직렬화 및 역직렬화

```rust
impl Keypair {
// 키페어를 64바이트 배열로 변환 (비밀키 32바이트 + 공개키 32바이트)
```

```
pub fn to_bytes(&self) -> [u8; 64] {
self.0.to_keypair_bytes()
}
// 바이트 배열에서 키페어 복원 (입력 데이터는 제로화됨)
pub fn try_from_bytes(kp: &mut [u8]) -> Result<Keypair, DecodingError> {
let bytes = <[u8; 64]>::try_from(&*kp)?;
ed25519::SigningKey::from_keypair_bytes(&bytes)
.map(|k| {
kp.zeroize(); // 보안을 위한 메모리 제로화
Keypair(k)
})
}
}
```

2. Noise 프로토콜 구현

2.1 Noise 프로토콜 개요

Noise는 libp2p의 주요 보안 계층으로, 다음과 같은 특징을 제공합니다:

- **핸드셰이크 패턴**: XX 패턴 사용 (상호 인증)
- **키 교환**: X25519 Diffie-Hellman
- **암호화**: ChaCha20-Poly1305 AEAD
- **해시**: SHA-256

2.2 Noise 설정 구조

```
pub struct Config {
dh_keys: AuthenticKeypair, // DH 키페어
params: NoiseParams, // Noise 프로토콜 파라미터
webtransport_certhashes: Option<HashSet<Multihash<64>>>, // WebTransport 인증서 ㅎ
prologue: Vec<u8>, // 핸드셰이크 프롤로그
}

// 인증된 키페어 구조
pub struct AuthenticKeypair {
pub keypair: Keypair, // X25519 DH 키페어
pub identity: KeypairIdentity, // 신원 키와 서명
}

pub struct KeypairIdentity {
pub public: identity::PublicKey, // 신원 공개키
pub signature: Vec<u8>, // DH 공개키에 대한 서명
}
```

2.3 X25519 키 교환 구현

```
impl Keypair {
```

```rust
// 새로운 X25519 키페어 생성
pub fn new() -> Keypair {
let mut sk_bytes = [0u8; 32];
rand::thread_rng().fill(&mut sk_bytes);
let sk = SecretKey(sk_bytes);
sk_bytes.zeroize();
Self::from(sk)
}
}

// X25519 비밀키에서 공개키 계산
impl From<SecretKey> for Keypair {
fn from(secret: SecretKey) -> Keypair {
let public = PublicKey(x25519(secret.0, X25519_BASEPOINT_BYTES));
Keypair { secret, public }
}
}

// Diffie-Hellman 공유 비밀 계산
impl snow::types::Dh for Keypair {
fn dh(&self, pk: &[u8], shared_secret: &mut [u8]) -> Result<(), snow::Error> {
let mut p = [0; 32];
p.copy_from_slice(&pk[..32]);
```

```
let ss = x25519(self.secret.0, p); // 공유 비밀 계산
shared_secret[..32].copy_from_slice(&ss[..]);
Ok(())
}
}
```

2.4 Noise XX 핸드셰이크 과정

XX 핸드셰이크는 3단계로 구성됩니다:

```
Initiator Responder
| |
| 1. -> e | (임시 공개키 전송)
| |
| 2. <- e, ee, s, es | (임시키+정적키 교환, 인증)
| |
| 3. -> s, se | (정적키 전송, 상호 인증 완료)
| |
```

구현 세부사항:

```
// Initiator 측 핸드셰이크
impl<T> OutboundConnectionUpgrade<T> for Config {
fn upgrade_outbound(self, socket: T, _: Self::Info) -> Self::Future {
async move {
```

```
let mut state = self.into_initiator(socket)?;
handshake::send_empty(&mut state).await?; // 1단계: 빈 페이로드
handshake::recv_identity(&mut state).await?; // 2단계: 상대방 신원 수신
handshake::send_identity(&mut state).await?; // 3단계: 자신의 신원 전송
let (pk, io) = state.finish()?;
Ok((pk.to_peer_id(), io))
}
}
}

// Responder 측 핸드셰이크
impl<T> InboundConnectionUpgrade<T> for Config {
fn upgrade_inbound(self, socket: T, _: Self::Info) -> Self::Future {
async move {
let mut state = self.into_responder(socket)?;
handshake::recv_empty(&mut state).await?; // 1단계: 빈 페이로드 수신
handshake::send_identity(&mut state).await?; // 2단계: 자신의 신원 전송
handshake::recv_identity(&mut state).await?; // 3단계: 상대방 신원 수신
let (pk, io) = state.finish()?;
Ok((pk.to_peer_id(), io))
}
}
```

```
}
```

2.5 신원 인증 메커니즘

```
// DH 키페어를 신원 키로 인증

impl Keypair {

pub fn into_authentic(

self,

id_keys: &identity::Keypair,

) -> Result<AuthenticKeypair, Error> {

// "noise-libp2p-static-key:" + DH 공개키에 대한 서명 생성

let sig = id_keys.sign(&[STATIC_KEY_DOMAIN.as_bytes(), self.public.as_ref()].con

let identity = KeypairIdentity {

public: id_keys.public(),

signature: sig,

};

Ok(AuthenticKeypair {

keypair: self,

identity,

})

}

}
```

```
// 핸드셰이크 완료 시 신원 검증

impl<T> State<T> {

pub fn finish(self) -> Result<(identity::PublicKey, Output<T>), Error> {

let id_pk = self.id_remote_pubkey.ok_or(Error::AuthenticationFailed)?;

// 서명 검증: "noise-libp2p-static-key:" + DH 공개키

let is_valid_signature = self.dh_remote_pubkey_sig.as_ref().is_some_and(|s| {

id_pk.verify(&[STATIC_KEY_DOMAIN.as_bytes(), pubkey.as_ref()].concat(), s)

});

if !is_valid_signature {

return Err(Error::BadSignature);

}

Ok((id_pk, Output::new(framed)))

}

}
```

3. TLS 구현

3.1 TLS 설정

libp2p는 표준 TLS 1.3을 대안 보안 계층으로 지원합니다:

```
// 클라이언트 TLS 설정

pub fn make_client_config(

keypair: &Keypair,
```

```
remote_peer_id: Option<PeerId>,

) -> Result<rustls::ClientConfig, certificate::GenError> {

let (certificate, private_key) = certificate::generate(keypair)?;

let mut crypto = rustls::ClientConfig::builder_with_provider(provider.into())

.with_protocol_versions(verifier::PROTOCOL_VERSIONS)

.dangerous()

.with_custom_certificate_verifier(Arc::new(

verifier::Libp2pCertificateVerifier::with_remote_peer_id(remote_peer_id),

))

.with_client_cert_resolver(cert_resolver);

crypto.alpn_protocols = vec![P2P_ALPN.to_vec()]; // "libp2p" ALPN

Ok(crypto)

}
```

3.2 libp2p TLS 인증서 생성

```
pub fn generate(

identity_keypair: &identity::Keypair,

) -> Result<(CertificateDer<'static>, PrivateKeyDer<'static>), GenError> {

// 인증서 서명용 임시 키페어 생성 (ECDSA P-256)

let certificate_keypair = rcgen::KeyPair::generate_for(&rcgen::PKCS_ECDSA_P256_S

let certificate = {

let mut params = rcgen::CertificateParams::default();
```

```
params.distinguished_name = rcgen::DistinguishedName::new();

// libp2p 확장 추가 (OID: 1.3.6.1.4.1.53594.1.1)

params.custom_extensions.push(make_libp2p_extension(

identity_keypair,

&certificate_keypair,

)?);

params.self_signed(&certificate_keypair)?

};

Ok((certificate.into(), rustls_key))

}
```

3.3 libp2p 확장 구조

```
// libp2p 확장 생성

fn make_libp2p_extension(

identity_keypair: &identity::Keypair,

certificate_keypair: &rcgen::KeyPair,

) -> Result<rcgen::CustomExtension, rcgen::Error> {

// "libp2p-tls-handshake:" + 인증서 공개키에 대한 서명

let signature = {

let mut msg = vec![];

msg.extend(P2P_SIGNING_PREFIX); // "libp2p-tls-handshake:"

msg.extend(certificate_keypair.public_key_der());
```

```
identity_keypair.sign(&msg)?

};

// ASN.1 인코딩: SignedKey ::= SEQUENCE { publicKey OCTET STRING, signature OCTET

let extension_content = {

let serialized_pubkey = identity_keypair.public().encode_protobuf();

yasna::encode_der(&(serialized_pubkey, signature))

};

let mut ext = rcgen::CustomExtension::from_oid_content(&P2P_EXT_OID, extension_c

ext.set_criticality(true); // 중요 확장으로 표시

Ok(ext)

}
```

4. 암호화 통신 계층

4.1 Noise 암호화 스트림

```
pub struct Output<T> {

io: Framed<T, Codec<snow::TransportState>>,

recv_buffer: Bytes,

recv_offset: usize,

send_buffer: Vec<u8>,

send_offset: usize,

}
```

```
impl<T: AsyncRead + Unpin> AsyncRead for Output<T> {

fn poll_read(

mut self: Pin<&mut Self>,

cx: &mut Context<'_>,

buf: &mut [u8],

) -> Poll<io::Result<usize>> {

// 암호화된 프레임 수신 및 복호화

loop {

if self.recv_buffer.len() > 0 {

let n = min(self.recv_buffer.len() - self.recv_offset, buf.len());

buf[..n].copy_from_slice(&self.recv_buffer[self.recv_offset..self.recv_offset +

self.recv_offset += n;

return Poll::Ready(Ok(n));

}

// 새로운 암호화된 프레임 수신

match Pin::new(&mut self.io).poll_next(cx) {

Poll::Ready(Some(Ok(frame))) => {

self.recv_buffer = frame; // 복호화된 데이터

self.recv_offset = 0;

}

Poll::Ready(None) => return Poll::Ready(Ok(0)),

Poll::Ready(Some(Err(e))) => return Poll::Ready(Err(e)),
```

```
Poll::Pending => return Poll::Pending,
}
}
}
}
```

4.2 프레임 기반 암호화

```
const MAX_FRAME_LEN: usize = 65535; // 최대 프레임 크기

impl<T: AsyncWrite + Unpin> AsyncWrite for Output<T> {
fn poll_write(
self: Pin<&mut Self>,
cx: &mut Context<'_>,
buf: &[u8],
) -> Poll<io::Result<usize>> {
let this = Pin::into_inner(self);
// 최대 프레임 크기에 도달하면 전송
if this.send_offset == MAX_FRAME_LEN {
ready!(Pin::new(&mut this.io).poll_ready(cx))?;
Pin::new(&mut this.io).start_send(&this.send_buffer)?; // 암호화 후 전송
this.send_offset = 0;
}
```

```
// 데이터를 송신 버퍼에 추가

let n = min(MAX_FRAME_LEN - this.send_offset, buf.len());

this.send_buffer[this.send_offset..this.send_offset + n].copy_from_slice(&buf[..

this.send_offset += n;

Poll::Ready(Ok(n))

}

}
```

5. 보안 특성 및 보장사항

5.1 Noise 프로토콜 보안 특성

- **전방향 보안성 (Forward Secrecy)**: 세션 키 노출 시에도 과거 통신 내용 보호
- **상호 인증**: 양방향 신원 확인
- **키 확인**: 공유 비밀의 무결성 보장
- **재생 공격 방지**: 핸드셰이크 해시를 통한 메시지 순서 보장

5.2 암호화 알고리즘 강도

X25519 키 교환:

- 128비트 보안 수준
- 타이밍 공격 저항성
- 작은 서브그룹 공격 방지

ChaCha20-Poly1305 AEAD:

- 256비트 키 크기

- 96비트 논스
- 128비트 인증 태그
- 고성능 소프트웨어 구현

SHA-256 해시:

- 256비트 출력
- 충돌 저항성
- 사전 이미지 저항성

5.3 키 관리 보안

```
impl Drop for SecretKey {
fn drop(&mut self) {
self.0.zeroize(); // 메모리에서 비밀키 안전하게 제거
}
}

// 키 직렬화 시 입력 데이터 제로화
pub fn try_from_bytes(kp: &mut [u8]) -> Result<Keypair, DecodingError> {
let result = ed25519::SigningKey::from_keypair_bytes(&bytes);
kp.zeroize(); // 성공 여부와 관계없이 입력 데이터 제로화
result.map(Keypair)
}
```

6. 성능 고려사항

6.1 핸드셰이크 성능

- **Noise XX**: 3 라운드트립, 최소 지연시간
- **Ed25519 서명**: 빠른 서명 생성 및 검증
- **X25519 DH**: 효율적인 키 교환

6.2 암호화 성능

- **ChaCha20**: CPU 집약적이지만 높은 처리량
- **프레임 기반 처리**: 버퍼링을 통한 효율성 향상
- **제로 카피**: 가능한 경우 메모리 복사 최소화

7. 확장성 및 호환성

7.1 프로토콜 네고시에이션

```
impl UpgradeInfo for Config {
type Info = &'static str;
fn protocol_info(&self) -> Self::InfoIter {
std::iter::once("/noise") // Noise 프로토콜 식별자
}
}
```

7.2 WebTransport 지원

```
// WebTransport 인증서 해시 확장
pub fn with_webtransport_certhashes(mut self, certhashes: HashSet<Multihash<64>>
self.webtransport_certhashes = Some(certhashes).filter(|h| !h.is_empty());
self
}


// 핸드셰이크 시 인증서 해시 검증
if let Some(expected_certhashes) = self.responder_webtransport_certhashes {
let received_certhashes = ext.webtransport_certhashes;
if !expected_certhashes.is_subset(&received_certhashes) {
return Err(Error::UnknownWebTransportCerthashes(
expected_certhashes,
received_certhashes,
));
}
}
```

8. 결론

Rust libp2p의 키 교환 및 암호화 메커니즘은 다음과 같은 특징을 제공합니다:

1. **강력한 보안**: 현대적 암호화 알고리즘과 프로토콜 사용
2. **성능 최적화**: 효율적인 구현과 최소한의 오버헤드

3. **유연성**: 다양한 키 타입과 프로토콜 지원

4. **상호 운용성**: 표준 프로토콜 준수

5. **메모리 안전성**: Rust의 안전성과 명시적 키 제로화

이러한 설계를 통해 libp2p는 P2P 네트워크에서 안전하고 효율적인 통신을 보장합니다.